

PROJECT 11 REPORT

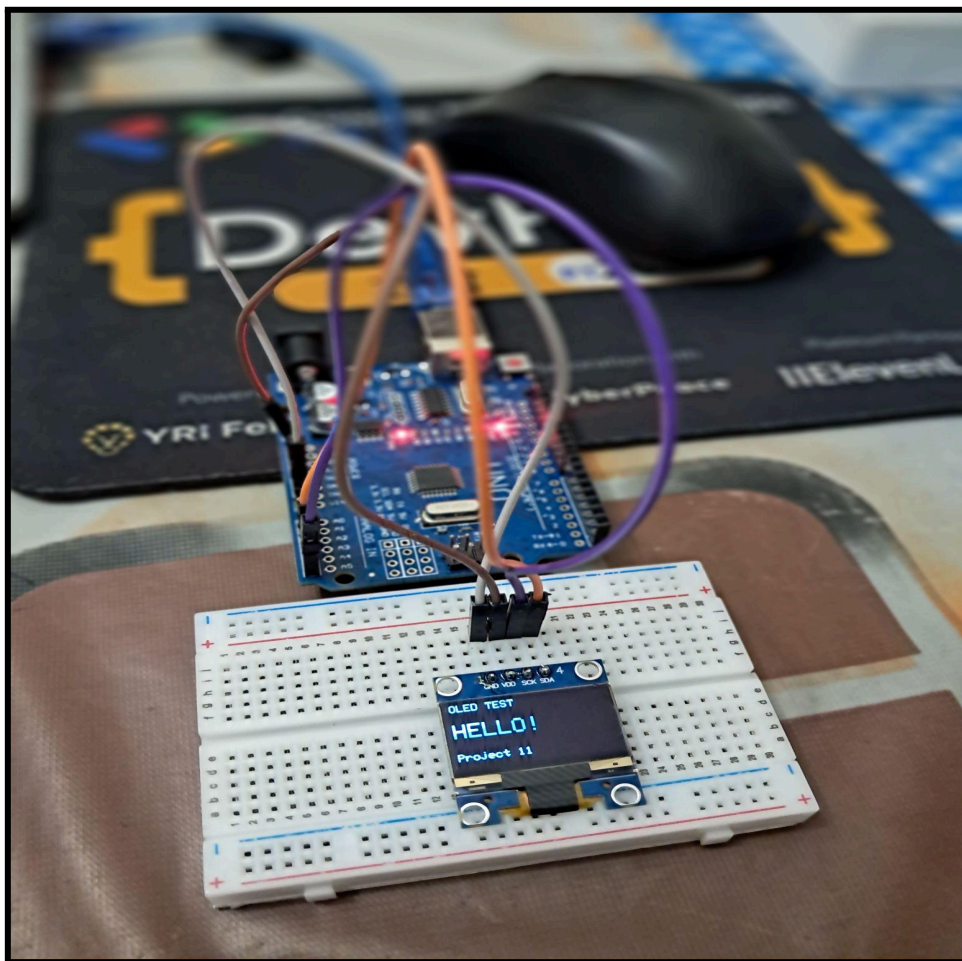
OLED Display Interfacing and Graphics Playground using Arduino

Name : Priyaansh Tiwari

Department : Electrical and Electronics Engineering

Date : August 2026

Duration : 3 days



1. Objective

The objective of this project was to interface a 0.96 inch OLED display with an Arduino and understand its basic operation, graphical drawing functions and animation.

The project was designed as an OLED playground to explore the display through progressively more complex experiments

2. Components Used

Component	Quantity
Arduino Uno R3	1
0.96 inch OLED Display	1
Breadboard	1
Jumper Wires	Multiple
USB Cable	1

3. Software Used

- Arduino IDE
- Adafruit GFX Library
- Adafruit SSD1306 Library
- Wire Library
- Wokwi
- Windows Laptop

4. Theory

The 0.96 inch OLED used in this project has a resolution of **128 × 64 pixels** and communicates with the Arduino using the I²C protocol.

The OLED module used four connections:

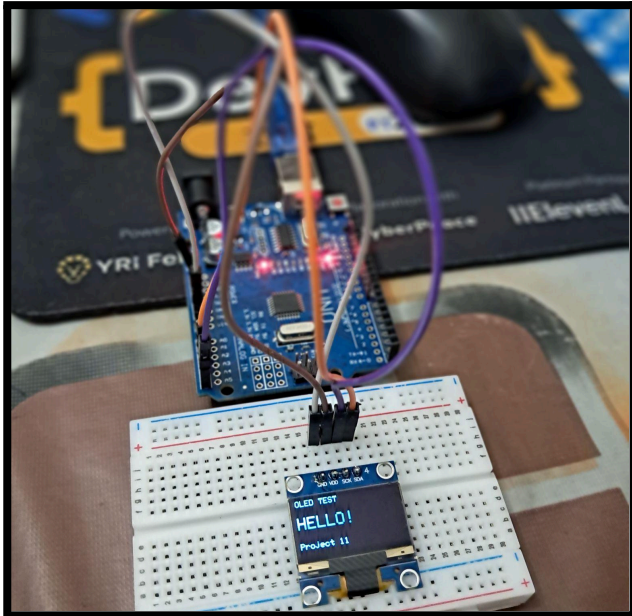
- GND – Ground
- VDD – Power supply
- SCK – I²C clock
- SDA – I²C data

The display was initialized using the Adafruit SSD1306 library:
display.begin(SSD1306_SWITCHCAPVCC, 0x3C);

The project introduced the concept of a display buffer. Drawing commands such as **drawPixel()**, **drawCircle()** and **drawRect()** which modify the buffer first and the contents are transferred to the actual OLED when: **display.display();** is called.

5. Tasks Performed

➤ Text Display



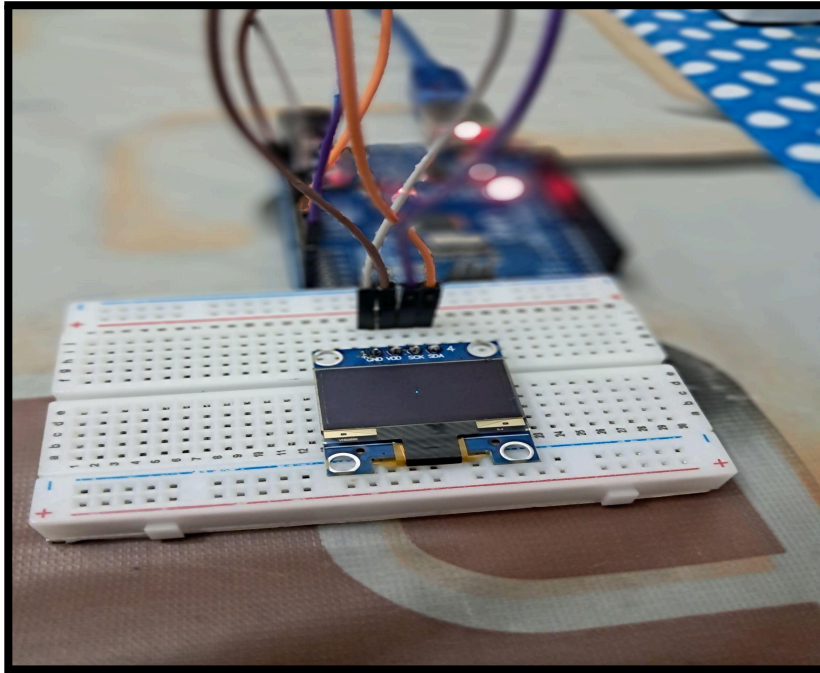
Different text display properties were explored as the text position was modified using OLED coordinates and different text sizes were tested.

➤ Graphics and Animation Playground

The OLED was then used as a graphical canvas.

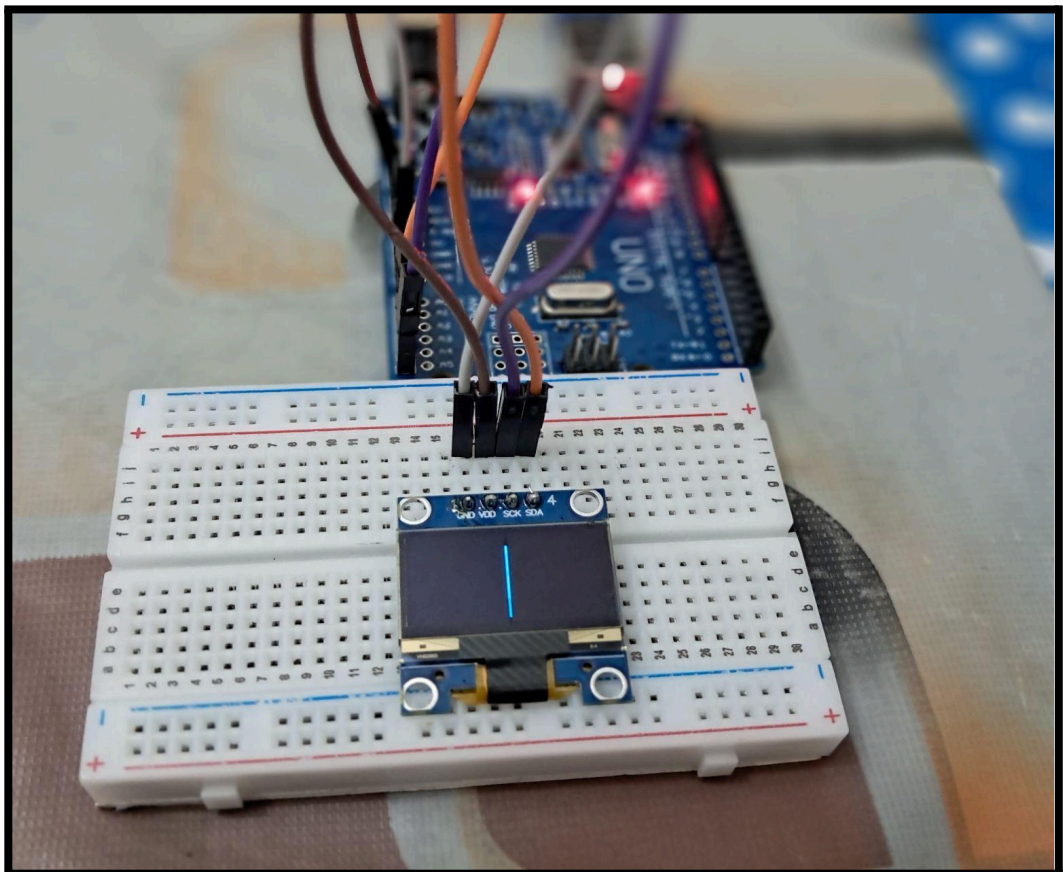
- **Pixel**

A single pixel was generated at a selected coordinate using: **display.drawPixel();**



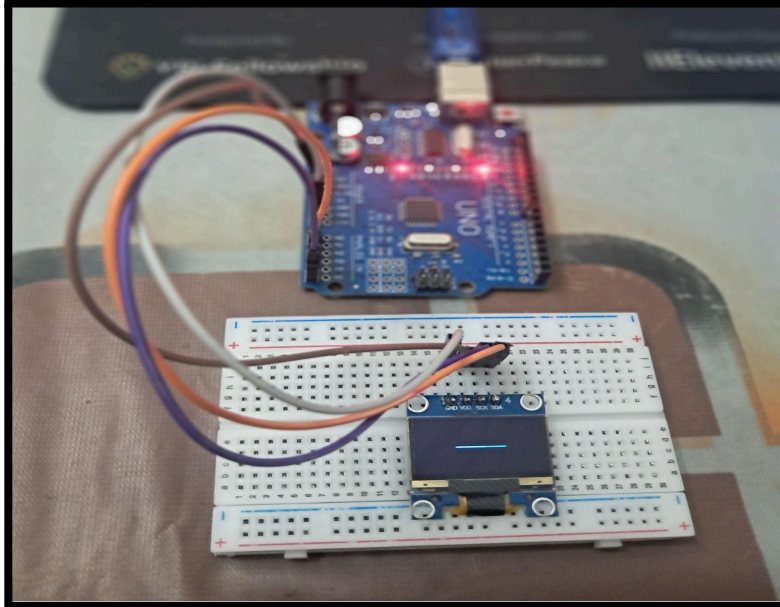
- **Vertical Line**

A vertical line was created by repeatedly drawing pixels with a for loop.



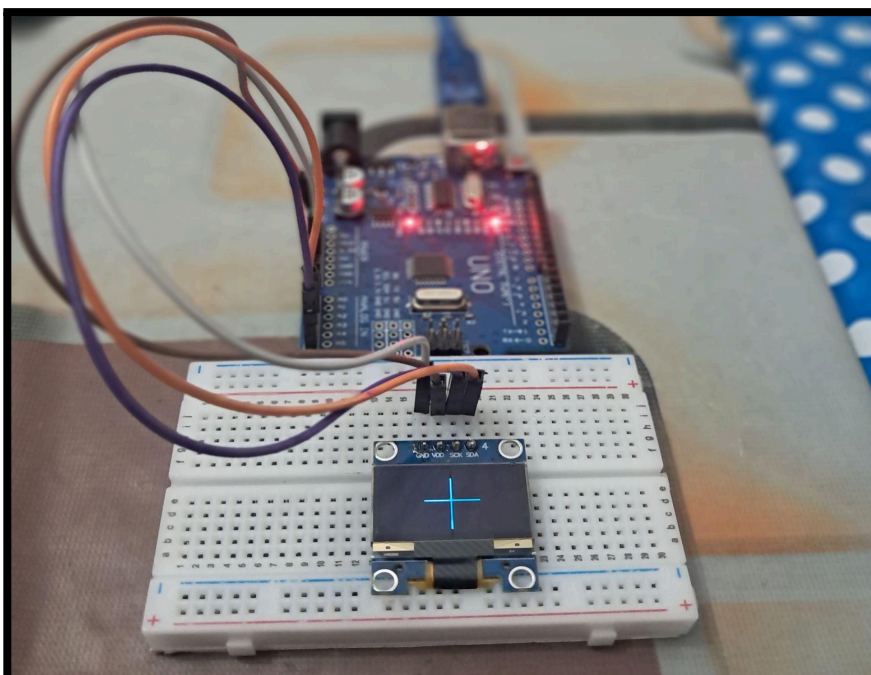
- **Horizontal Line**

A horizontal line was created by changing the X coordinate inside a loop.



- **Plus**

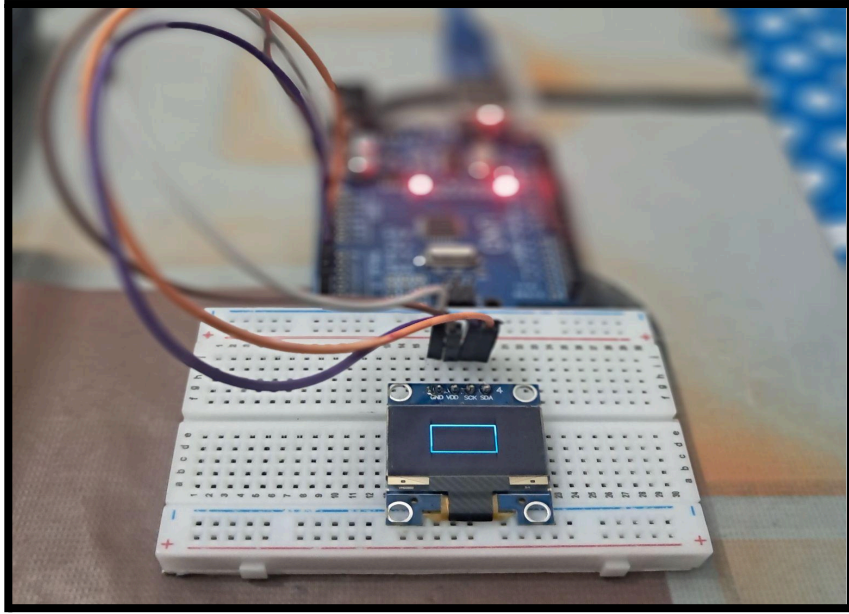
Vertical and horizontal lines were combined to create a plus shape, multiple graphical elements can be drawn into the buffer before calling `display.display()`



- **Rectangle**

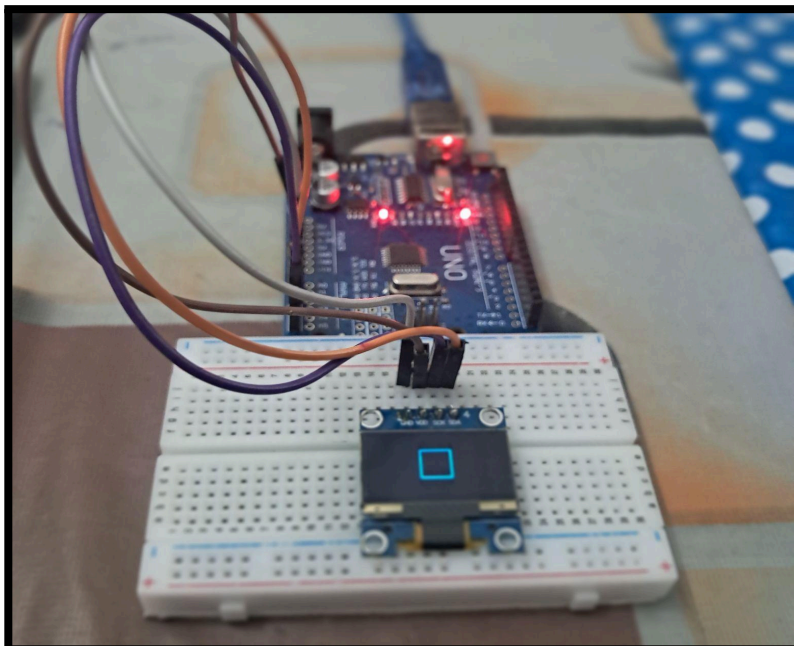
A rectangle was created using: **display.drawRect()**;

The parameters used represented the X position, Y position, width, height, and colour.



- **Square**

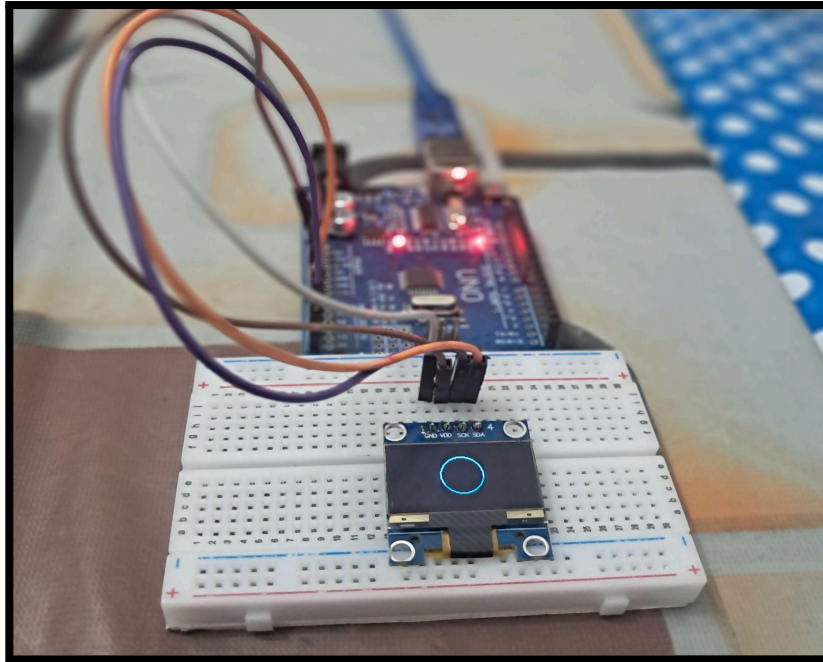
A square was created using **drawRect()** with equal width and height.



- **Circle**

A circle was created using: **display.drawCircle(64, 32, 20, SSD1306_WHITE);**

The value 20 represents the radius of the circle.



- **Moving Pixel/Circle**

A small filled circle was moved across the OLED using a changing X coordinate.



Video

● Bouncing Ball

A bouncing-ball animation was developed using:

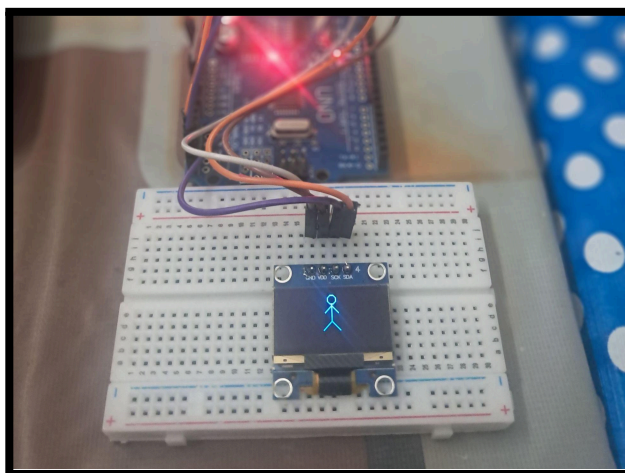
- Position variable
- Direction variable
- Boundary checking



Video

● Stickman and Walking Animation

A basic stickman was created using the OLED's graphical drawing functions having Heads,Body,Hands,Legs.



Video

A bitmap/frame-based walking animation was also tested to understand how more complex animations can be displayed on the OLED.

6. Learning Outcomes

During this project, the following concepts were learned:

- OLED Display Interfacing
- I²C Communication
- I²C Device Address
- OLED Resolution
- Pixel Coordinates
- Display Buffer
- `display.clearDisplay()`
- `display.display()`
- `drawPixel()`
- `drawLine()`
- `drawRect()`
- `fillRect()`
- `drawCircle()`
- `fillCircle()`
- Animation using `loop()`
- Basic State Based Animation
- Frame Based Animation
- Bitmap Based OLED Animation

7. Problems Faced

- I. Initially, the purpose of `display.display()` was not completely clear. (**`display.display()` sends the buffer contents to the OLED**)

- II. The first moving object animation did not work correctly because the position and direction variables were declared inside the loop() function.**(The variables were moved outside loop() so their values could persist between iterations)**

8. Observations

- The OLED successfully communicated with the Arduino through I²C.
- The OLED used the 0x3C I²C address.
- The 128 × 64 display provided direct pixel level control.
- Basic geometric shapes could be created using Adafruit GFX functions.
- Multiple objects could be prepared in the display buffer before updating the OLED.
- loop() can be used to continuously update the display for animations.
- Position and direction variables are important for maintaining animation state.
- More complex animations can be created using multiple frames or bitmap data.
- The OLED can be used as a small graphical interface rather than only displaying text.

9. Applications and Future Scope

The concepts learned in this project are used in Embedded System Displays, Sensor Dashboards, Robotics Interfaces and Graphical Indicator.

Possible future upgrades include Real time system status, Menus and user interaction and More complex embedded applications.

10. Conclusion

This project successfully introduced OLED display interfacing and graphical programming using Arduino.

The project progressed from basic OLED initialization and text display to pixel level graphics, geometric shapes, moving objects, a bouncing ball and a walking animation.

The project provided the required foundation for using OLED displays in more complex embedded systems